

GANs Lab: Experiment 3: DCGANs

Name: Aparna Iyer

PRN: 22070126017

Batch: 2022-2026

Branch: AI-ML A1



APARNA.IYER BTECH2022 · 8H AGO · PRIVATE



DCGAN_GANsLab_Experiment3



Notebook Input Output Logs Comments (0)

In [1]:

```
# Import required libraries
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.utils as vutils
import matplotlib.pyplot as plt
import numpy as np
import os

# Check for GPU availability
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cuda

Loading and Preprocessing the CelebA Image Dataset.

In [2]:

```
# Define transformations: Resize images, normalize to [-1,1]
transform = transforms.Compose([
    transforms.Resize(64),
    transforms.CenterCrop(64),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize to [-1,1]
])

# Load CelebA dataset
dataset = torchvision.datasets.ImageFolder(root="/kaggle/input/celeba-dataset", transform=transform)

# DataLoader with GPU optimization
dataloader = torch.utils.data.DataLoader(dataset, batch_size=128, shuffle=True,
                                         num_workers=4, pin_memory=True)

# Move batch of images to GPU
real_batch = next(iter(dataloader))
real_images = real_batch[0].to(device) # Move images to GPU

# Display a batch of images
plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Training Images")
plt.imshow(np.transpose(vutils.make_grid(real_images[:64].cpu(), padding=2, normalize=True), (1,2,0)))
plt.show()
```

Version 1 of 1



Runtime

▶ 7h 34m 6s · GPU T4 ×2

Input

DATASETS

celeba-dataset

Tags

GPU

Language

Python

Training Images



Define the Generator Neural Network.

```
In [3]: class Generator(nn.Module):
        """Generator Model for DCGAN"""
        def __init__(self):
            super(Generator, self).__init__()
            self.main = nn.Sequential(
                nn.ConvTranspose2d(100, 512, 4, 1, 0, bias=False),
                nn.BatchNorm2d(512),
                nn.ReLU(True),

                nn.ConvTranspose2d(512, 256, 4, 2, 1, bias=False),
                nn.BatchNorm2d(256),
                nn.ReLU(True),

                nn.ConvTranspose2d(256, 128, 4, 2, 1, bias=False),
                nn.BatchNorm2d(128),
                nn.ReLU(True),

                nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
                nn.BatchNorm2d(64),
                nn.ReLU(True),

                nn.ConvTranspose2d(64, 3, 4, 2, 1, bias=False),
                nn.Tanh()
            )

        def forward(self, input):
            return self.main(input)

# Initialize Generator (Instantiation)
netG = Generator().to(device)
```

Define the Discriminator Neural Network

```
In [4]: class Discriminator(nn.Module):
        """Discriminator Model for DCGAN"""
        def __init__(self):
            super(Discriminator, self).__init__()
            self.main = nn.Sequential(
                nn.Conv2d(3, 64, 4, 2, 1, bias=False),
                nn.LeakyReLU(0.2, inplace=True),

                nn.Conv2d(64, 128, 4, 2, 1, bias=False),
                nn.BatchNorm2d(128),
                nn.LeakyReLU(0.2, inplace=True),

                nn.Conv2d(128, 256, 4, 2, 1, bias=False),
                nn.BatchNorm2d(256),
                nn.LeakyReLU(0.2, inplace=True),

                nn.Conv2d(256, 512, 4, 2, 1, bias=False),
                nn.BatchNorm2d(512),
                nn.LeakyReLU(0.2, inplace=True),

                nn.Conv2d(512, 1, 4, 1, 0, bias=False),
                nn.Sigmoid()
            )

        def forward(self, input):
            return self.main(input)

        # Initialize Discriminator (Instantiation)
        netD = Discriminator().to(device)
```

Define Loss Function and Optimizers.

```
In [5]: #Define the Loss Function and Optimizers for Generator and Discriminator Networks.
        criterion = nn.BCELoss()
        optimizerG = optim.Adam(netG.parameters(), lr=0.0002, betas=(0.5, 0.999))
        optimizerD = optim.Adam(netD.parameters(), lr=0.0002, betas=(0.5, 0.999))
```

Train the DCGAN.

```
In [6]: #Define and set the Hyperparameters.

        num_epochs = 50 # Number of training iterations over the dataset
        batch_size = 128 # Number of images per batch
        learning_rate = 0.0002 # Learning rate for both Generator & Discriminator
        beta1 = 0.5 # Beta1 hyperparameter for Adam optimizer (recommended by DCGAN paper)
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # Use GPU if available
        latent_dim = 100 # Size of random noise vector (z)
```

```
In [7]:
for epoch in range(num_epochs):
    for i, (real_images, _) in enumerate(dataloader):
        real_images = real_images.to(device)
        batch_size = real_images.size(0)

        # Correct label shapes
        real_labels = torch.ones(batch_size, device=device) # [128]
        fake_labels = torch.zeros(batch_size, device=device) # [128]

        # -----
        # Train Discriminator
        # -----
        netD.zero_grad()

        # Train with real images
        output_real = netD(real_images).view(-1) # Shape [128]
        loss_real = criterion(output_real, real_labels) # Shape [128]
        loss_real.backward()

        # Train with fake images
        noise = torch.randn(batch_size, 100, 1, 1, device=device)
        fake_images = netG(noise)
        output_fake = netD(fake_images.detach()).view(-1) # Shape [128]
        loss_fake = criterion(output_fake, fake_labels) # Shape [128]
        loss_fake.backward()

        optimizerD.step() # Update Discriminator weights
```

```
In [8]:
import torch
import matplotlib.pyplot as plt
import torchvision.utils as vutils

for epoch in range(num_epochs):
    for i, (real_images, _) in enumerate(dataloader):
        real_images = real_images.to(device)
        batch_size = real_images.size(0)

        # Correct label shapes
        real_labels = torch.ones(batch_size, device=device)
        fake_labels = torch.zeros(batch_size, device=device)

        # -----
        # Train Discriminator
        # -----
        netD.zero_grad()

        # Train with real images
        output_real = netD(real_images).view(-1)
        loss_real = criterion(output_real, real_labels)
        loss_real.backward()

        # Train with fake images
        noise = torch.randn(batch_size, 100, 1, 1, device=device)
        fake_images = netG(noise)
        output_fake = netD(fake_images.detach()).view(-1)
        loss_fake = criterion(output_fake, fake_labels)
        loss_fake.backward()
```

```

optimizerD.step()

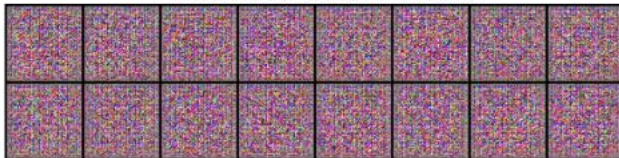
# -----
# Train Generator
# -----
netG.zero_grad()
output_fake_for_G = netD(fake_images).view(-1)
loss_G = criterion(output_fake_for_G, real_labels)
loss_G.backward()
optimizerG.step()

# Visualization every 500 batches
if i % 500 == 0:
    with torch.no_grad():
        noise = torch.randn(16, 100, 1, 1, device=device) # Generate 16 images for visualization
        fake_images = netG(noise).cpu() # Move to CPU for visualization

        plt.figure(figsize=(8, 8))
        plt.axis("off")
        plt.title(f"Generated Images at Epoch {epoch}, Batch {i}")
        plt.imshow(
            utils.make_grid(fake_images, padding=2, normalize=True).permute(1, 2, 0)
        ) # Convert to correct format for visualization
        plt.show()

```

Generated Images at Epoch 0, Batch 0



Generated Images at Epoch 0, Batch 500



Generated Images at Epoch 0, Batch 1000



Generated Images at Epoch 0, Batch 1500



Generated Images at Epoch 15, Batch 500



Generated Images at Epoch 15, Batch 1000



Generated Images at Epoch 20, Batch 0



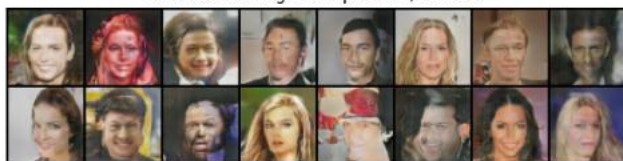
Generated Images at Epoch 20, Batch 500



Generated Images at Epoch 20, Batch 1000



Generated Images at Epoch 35, Batch 0



Generated Images at Epoch 35, Batch 500



Generated Images at Epoch 35, Batch 1000



Generated Images at Epoch 49, Batch 500



Generated Images at Epoch 49, Batch 1000



Generated Images at Epoch 49, Batch 1500



In [9]:

```
#Visualize the generated images
import torch
import torchvision.utils as vutils
import matplotlib.pyplot as plt

# Set your model to evaluation mode
netG.eval()

# Generate a batch of fake images
latent_dim = 100 # Same as used in training
num_samples = 64 # Number of images to generate
fixed_noise = torch.randn(num_samples, latent_dim, 1, 1, device=device) # Noise input

with torch.no_grad():
    fake_images = netG(fixed_noise).detach().cpu() # Generate images

# Visualize the images
plt.figure(figsize=(8,8))
plt.axis("off")
plt.title("Generated Images")
plt.imshow(np.transpose(vutils.make_grid(fake_images, padding=2, normalize=True), (1,2,0)))
plt.show()
```

Generated Images

